



11. Animación de un personaje 2D

Física e Inteligencia Artificial para Videojuegos

Doble Grado en Física e Inteligencia Artificial

Curso 2025 – 2026

Fernando Madrid Recio

El Componente Animator

El componente **Animator** es el encargado de gestionar las animaciones de un GameObject en Unity. Actúa como puente entre el código y el **Animator Controller** (la máquina de estados que define cuándo y cómo se reproducen las animaciones).

Animator Controller

Archivo que contiene la máquina de estados de animaciones. Define los estados (clips) y las transiciones entre ellos.

Animation Clip

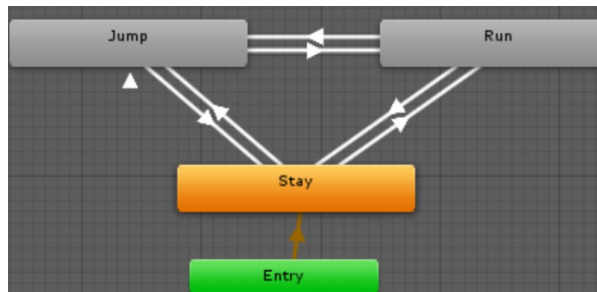
Cada animación concreta (idle, run, jump...). Se asigna a un estado dentro del Animator Controller.

Parámetros

Variables (Trigger, Bool, Int, Float) que el código modifica para activar transiciones entre estados.

Animator Controller: Estructura

El Animator Controller es una máquina de estados visual donde cada nodo es un Animation Clip y las flechas son transiciones condicionadas por parámetros.



Acceso desde código (C#):

```
// Obtener referencia al componente Animator
private Animator anim;
void Start()
{
    anim = GetComponent<Animator>();
}
```

Parámetro tipo Trigger

Un **Trigger** es un parámetro de tipo pulsación: se activa una sola vez y se resetea automáticamente tras disparar la transición. Es preferible para animaciones que deben ejecutarse un **número determinado de veces** (p.ej. un salto, un ataque, una muerte).

Uso desde código:

```
// Activar el Trigger "Jump"  
anim.SetTrigger("Jump");  
  
// Cancelar (si se necesita)  
anim.ResetTrigger("Jump");
```

Características clave:

- Se activa con SetTrigger() y se consume automáticamente al usarse.
- No tiene valor persistente: solo está "pendiente" o "consumido", nunca true/false.
- Ideal para: saltos, ataques, golpes recibidos, recoger objetos.
- Si varias transiciones lo pueden consumir, Unity usa la primera válida.

Parámetros tipo Bool

Un **Bool** es un parámetro que mantiene un valor **true/false de forma persistente**. Unity evalúa su valor en cada frame para decidir si mantiene o cambia el estado activo. Es preferible para animaciones de **duración indeterminada** (p.ej. correr, agacharse, estar en el agua).

Uso desde código:

```
// Activar la animación de correr  
anim.SetBool("isRunning", true);  
  
// Desactivarla al soltar la tecla  
anim.SetBool("isRunning", false);
```

Características clave:

- Persiste hasta que se cambie explícitamente con SetBool().
- La transición de vuelta requiere condición contraria (p.ej. isRunning = false).
- Ideal para: correr, agacharse, nadar, apuntar, flotar.
- Usa GetBool() para leer el valor desde código si es necesario.

Trigger vs Bool: ¿Cuándo usar cada uno?

Trigger	Bool
Pulsación única (one-shot)	Estado persistente (on/off)
Se consume al dispararse	Permanece hasta cambiarlo
No tiene valor true/false	Tiene valor true o false
Salto, ataque, golpe, muerte	Correr, agacharse, nadar
SetTrigger() / ResetTrigger()	SetBool() / GetBool()

Has Exit Time

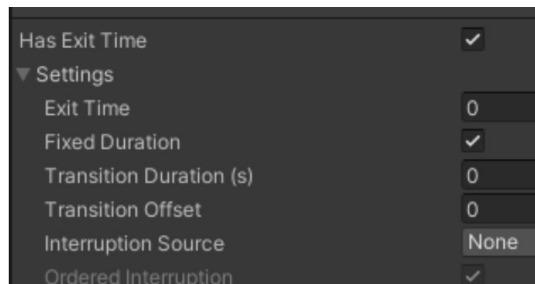
Has Exit Time es una opción de cada transición en el Animator Controller. Cuando está activada, Unity **espera a que el clip actual llegue a un porcentaje de su duración** (Exit Time) antes de evaluar si la condición de transición se cumple.

Has Exit Time: ACTIVADO

- La transición no ocurre hasta que el clip actual llega al Exit Time definido.
- Útil para animaciones que deben completarse (p.ej. animación de muerte).
- **Problema:** introduce retardo perceptible. Pulsar "saltar" puede no responder inmediatamente si el clip idle no ha llegado al Exit Time.
- EVÍTALO en animaciones de movimiento **reactivo** (correr, saltar, atacar) donde la respuesta debe ser **inmediata**.

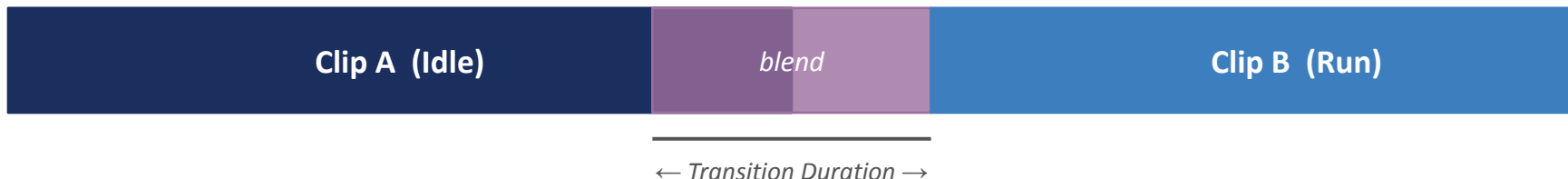
Has Exit Time: DESACTIVADO

- La transición ocurre en cuanto la condición se cumple, **sin esperar al clip**.
- Respuesta inmediata al input del jugador.
- La condición (Trigger o Bool) es el único factor que controla cuándo transiciona.



Transition Duration

Transition Duration define el tiempo de **blend (mezcla)** entre la animación saliente y la entrante. Durante este tiempo se interpolan los frames de ambos clips para crear una transición suave. Recomendado **sólo** para 3D



Ejemplo: Personaje 2D

Script que controla las animaciones de un personaje 2D con Idle, Run y Jump:

```
private Animator anim;
private Rigidbody2D rb;
bool isGrounded;

void Start()
{
    anim = GetComponent<Animator>();
    rb = GetComponent<Rigidbody2D>();
}

void Update()
{
    float h = Input.GetAxis("Horizontal");
    anim.SetBool("isRunning", h != 0); // Bool: duración indeterminada

    if (Input.GetKeyDown(KeyCode.Space) && isGrounded)
    {
        anim.SetTrigger("Jump"); // Trigger: ocurre una vez
        rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);
    }
}
```